

UNIVERSITÀ DEGLI STUDI DI ROMA TRE
DIPARTIMENTO DI INGEGNERIA CIVILE, INFORMATICA E DELLE
TECNOLOGIE AERONAUTICHE



PROGETTO 2 PER "TECNOLOGIE E ARCHITETTURE PER LA
GESTIONE DEI DATI"

Analisi del comportamento dei livelli di Isolamento in PostgreSQL

Autore
Luca Borrelli

Matricola
559443

Sommario

Il progetto si propone di studiare e sperimentare il comportamento di PostgreSQL nella gestione dei livelli di isolamento, con particolare riferimento alle principali anomalie che possono verificarsi durante l'esecuzione concorrente di transazioni.

L'obiettivo principale è quello di analizzare come i diversi livelli di isolamento influenzino il comportamento del database in presenza di operazioni concorrenti, evidenziando le garanzie offerte da ciascun livello e le potenziali anomalie che possono manifestarsi.

Per maggiori dettagli, il codice sorgente del progetto è disponibile al seguente link GitHub, cliccando sull'icona si viene reindirizzati direttamente al repository.



https://github.com/lukebo01/Progetto2_bd2

Indice

1	Ambiente di lavoro e struttura dei dati	1
1.1	Ambiente di sviluppo	1
1.2	Struttura della tabella	1
1.3	Generazione dei dati	1
2	Livelli di isolamento in PostgreSQL	3
2.1	Introduzione al Controllo della Concorrenza in PostgreSQL	3
2.2	Livello READ COMMITTED	3
2.2.1	Come Funziona	3
2.2.2	Anomalie Prevenute e Consentite	3
2.2.3	Caso d'Uso Tipico	4
2.3	Livello REPEATABLE READ	4
2.3.1	Come Funziona	4
2.3.2	Anomalie Prevenute e Consentite	4
2.3.3	Caso d'Uso Tipico	4
2.4	Livello SERIALIZABLE	5
2.4.1	Come Funziona	5
2.4.2	Anomalie Prevenute	5
2.5	Tabella Riassuntiva	5
3	Test implementati	6
3.1	Test 1: Inserimento con cicli di lettura/scrittura	6
3.1.1	Obiettivo	6
3.1.2	Procedura	6
3.1.3	Risultati attesi	6
3.1.4	Risultati attesi	6
3.2	Test 2: Modifica concorrentiale dello stesso campo	7
3.2.1	Obiettivo	7
3.2.2	Procedura	7
3.2.3	Risultati attesi	7
3.3	Test 3: Inserimento di un campo chiave primaria già esistente	7
3.3.1	Obiettivo	7
3.3.2	Procedura	7
3.3.3	Risultati attesi	8
4	Implementazione	9
4.1	Connessione al database	9
4.2	Implementazione del Test 1	9
4.3	Implementazione del Test 2	10
4.4	Implementazione del Test 3	12
5	Risultati ottenuti	15
5.1	Risultati del Test 1	15
5.1.1	READ COMMITTED	15

5.1.2	REPEATABLE READ	15
5.1.3	SERIALIZABLE	15
5.2	Risultati del Test 2	16
5.2.1	READ COMMITTED	16
5.2.2	REPEATABLE READ	16
5.2.3	SERIALIZABLE	16
5.3	Risultati del Test 3	17
5.3.1	READ COMMITTED	17
5.3.2	REPEATABLE READ	17
5.3.3	SERIALIZABLE	17
6	Analisi dei risultati	19
6.1	Test 1: Inserimento con cicli di lettura/scrittura	19
6.2	Test 2: Modifica concorrentiale dello stesso campo	19
6.3	Test 3: Inserimento di un campo chiave primaria già esistente	19
7	Conclusioni	21
7.1	Differenze tra i livelli di isolamento	21
7.2	Comportamento di lock in PostgreSQL	21
7.3	Considerazioni per le applicazioni reali	21

Elenco delle figure

Elenco delle tabelle

2.1	Anomalie di concorrenza nei livelli di isolamento di PostgreSQL.	5
-----	--	---

1 Ambiente di lavoro e struttura dei dati

1.1 Ambiente di sviluppo

Per la realizzazione di questo progetto è stato utilizzato:

- PostgreSQL 16.x come sistema di gestione del database
- Python 3.x come linguaggio di programmazione per i test
- Libreria psycopg2 per la connessione a PostgreSQL da Python
- Libreria Faker per la generazione di dati casuali
- Jupyter Notebook come ambiente di sviluppo interattivo

1.2 Struttura della tabella

Per la sperimentazione è stata utilizzata una tabella *Studente* con la seguente struttura:

```
1 CREATE TABLE Studente (  
2     matricola VARCHAR(10) PRIMARY KEY,  
3     nome VARCHAR(100) NOT NULL,  
4     cognome VARCHAR(100) NOT NULL,  
5     cfu INTEGER NOT NULL  
6 );
```

La tabella rappresenta un semplice archivio di studenti universitari, dove:

- **matricola** rappresenta l'identificativo univoco dello studente (chiave primaria)
- **nome** e **cognome** sono i dati anagrafici
- **cfu** rappresenta i Crediti Formativi Universitari accumulati dallo studente

1.3 Generazione dei dati

Per popolare la tabella sono stati generati 50 studenti con dati casuali utilizzando la libreria Faker. Il codice per la generazione è il seguente:

```
1 # Generatore di dati casuali  
2 fake = Faker('it_IT')  
3  
4 # Funzione per generare dati casuali per gli studenti  
5 def generate_students(num_students):  
6     students = []  
7     for i in range(1, num_students + 1):  
8         matricola = f"{10000 + i}"  
9         nome = fake.first_name()
```

```
10     cognome = fake.last_name()
11     cfu = random.randint(0, 180)
12     students.append((matricola, nome, cognome, cfu))
13     return students
14
15 # Inserisco 50 studenti casuali
16 students = generate_students(50)
17 cursor1.executemany(
18     "INSERT INTO Studente (matricola, nome, cognome, cfu) VALUES (%s, %s
19     , %s, %s)",
20     students
21 )
```

Ogni studente ha:

- Una matricola sequenziale a partire da 10001
- Nome e cognome generati casualmente (con nomi italiani)
- Un numero di CFU casuale tra 0 e 180

2 Livelli di isolamento in PostgreSQL

2.1 Introduzione al Controllo della Concorrenza in PostgreSQL

PostgreSQL gestisce l'accesso concorrente ai dati attraverso un meccanismo sofisticato chiamato **Multiversion Concurrency Control (MVCC)**. A differenza dei sistemi basati su lock di lettura, in PostgreSQL le operazioni di lettura non bloccano mai le operazioni di scrittura e viceversa. Questo è possibile perché il sistema mantiene più *versioni* di una stessa riga, permettendo a ogni transazione di operare su una "fotografia" consistente del database, chiamata *snapshot*.

Il livello di isolamento di una transazione definisce quale snapshot viene utilizzato e per quanto tempo rimane valido, proteggendo la transazione da diverse anomalie di concorrenza. Analizziamo i tre livelli principali offerti da PostgreSQL: **READ COMMITTED**, **REPEATABLE READ** e **SERIALIZABLE**.

2.2 Livello READ COMMITTED

Questo è il livello di isolamento di **default** in PostgreSQL. Offre un buon bilanciamento tra prestazioni e consistenza dei dati.

2.2.1 Come Funziona

Una transazione che opera a livello **READ COMMITTED** utilizza uno **snapshot diverso per ogni istruzione**.

- Quando un'istruzione **SELECT** (o **UPDATE**, **DELETE**) inizia, viene creato uno snapshot del database.
- L'istruzione vede solo i dati che erano stati salvati (committati) *prima* dell'inizio dell'istruzione stessa.
- Una volta che l'istruzione è completata, lo snapshot viene scartato.
- L'istruzione successiva nella stessa transazione creerà un nuovo snapshot, aggiornato allo stato corrente del database.

2.2.2 Anomalie Prevenute e Consentite

- **Previene i *Dirty Reads* (Letture Sporche):** Poiché ogni snapshot contiene solo dati committati, è impossibile leggere modifiche non ancora salvate da altre transazioni.
- **Consente i *Non-Repeatable Reads* (Letture Non Ripetibili):** Se una transazione esegue la stessa query due volte, potrebbe ottenere risultati diversi. Questo accade perché una seconda transazione potrebbe aver modificato i dati e fatto commit tra la prima e la seconda esecuzione della query. Ogni query usa un nuovo snapshot e vedrà quindi la modifica.

- **Consente i *Phantom Reads* (Letture Fantasma):** Similmente al punto precedente, se una seconda transazione inserisce nuove righe che soddisfano i criteri di una query, una successiva esecuzione della stessa query le vedrà, poiché opererà su un nuovo snapshot che include le nuove righe.

2.2.3 Caso d'Uso Tipico

È ideale per la maggior parte delle operazioni OLTP (Online Transaction Processing), come brevi letture e scritture, dove non è strettamente necessario che i dati rimangano immutati per tutta la durata di una transazione multi-istruzione.

2.3 Livello REPEATABLE READ

Questo livello offre garanzie di consistenza più forti rispetto a `READ COMMITTED`, al costo di un potenziale aumento dei conflitti tra transazioni.

2.3.1 Come Funziona

Una transazione che opera a livello `REPEATABLE READ` utilizza un **unico snapshot per tutta la sua durata**.

- Quando la **prima istruzione** della transazione inizia, viene creato uno snapshot del database.
- Tutte le istruzioni successive all'interno della stessa transazione (letture e scritture) utilizzeranno sempre e solo questo snapshot iniziale.
- La transazione opera quindi su una visione del database "congelata" nel tempo, ignorando completamente qualsiasi modifica committata da altre transazioni dopo il suo inizio.

2.3.2 Anomalie Prevenute e Consentite

- **Previene i *Non-Repeatable Reads*:** Eseguendo la stessa query più volte si otterranno sempre gli stessi risultati, perché lo snapshot di riferimento è fisso.
- **Previene (nella maggior parte dei casi) i *Phantom Reads*:** Poiché lo snapshot è fisso, nuove righe inserite da altre transazioni non saranno visibili.
- **Può fallire con un errore di *serialization failure*:** Se la nostra transazione cerca di modificare una riga che è stata modificata e committata da un'altra transazione concorrente (dopo l'inizio del nostro snapshot), PostgreSQL rileverà un conflitto. La nostra transazione verrà terminata (con un `ROLLBACK`) e il client riceverà un errore. Sarà compito dell'applicazione tentare nuovamente l'intera transazione.

2.3.3 Caso d'Uso Tipico

È utile per transazioni complesse che eseguono più query e necessitano di operare su una visione stabile dei dati, come la generazione di report o operazioni che coinvolgono logiche di business complesse su più tabelle.

2.4 Livello *SERIALIZABLE*

Questo è il livello di isolamento più forte, che garantisce che l'esecuzione concorrente di un gruppo di transazioni produca lo stesso risultato che si otterrebbe eseguendole una dopo l'altra, in un qualche ordine seriale.

2.4.1 Come Funziona

Il livello *SERIALIZABLE* in PostgreSQL si basa su una tecnica avanzata chiamata **Serializable Snapshot Isolation (SSI)**. Come per *REPEATABLE READ*, utilizza un unico snapshot per tutta la durata della transazione. Tuttavia, aggiunge un meccanismo di monitoraggio molto più sofisticato.

- La transazione opera su uno snapshot fisso.
- Mentre la transazione è in esecuzione, PostgreSQL tiene traccia delle dipendenze tra le transazioni concorrenti. Monitora se le letture di una transazione sono in conflitto con le scritture di un'altra.
- Al momento del **COMMIT**, PostgreSQL esegue un controllo di validazione. Se rileva che l'esecuzione concorrente potrebbe aver portato a un risultato anomalo (che non sarebbe potuto accadere in un'esecuzione seriale), una delle transazioni viene scelta per fallire.
- La transazione che fallisce riceve un errore di **serialization failure** e deve essere ritentata dall'applicazione.

2.4.2 Anomalie Prevenute

- **Previene tutte le anomalie note**, inclusi i *Phantom Reads* e altre anomalie più sottili come la *Write Skew*. Garantisce una consistenza totale dei dati, come se ogni transazione fosse l'unica in esecuzione sul sistema.

2.5 Tabella Riassuntiva

La seguente tabella riassume le proprietà dei tre livelli di isolamento in PostgreSQL.

Anomalia	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Dirty Read	Non possibile	Non possibile	Non possibile
Non-Repeatable Read	Possibile	Non possibile	Non possibile
Phantom Read	Possibile	Non possibile	Non possibile
Serialization Anomaly (es. Write Skew)	Possibile	Possibile	Non possibile

Tabella 2.1: Anomalie di concorrenza nei livelli di isolamento di PostgreSQL.

3 Test implementati

Per verificare il comportamento dei diversi livelli di isolamento sono stati implementati tre test specifici, ciascuno progettato per evidenziare particolari aspetti della concorrenza.

3.1 Test 1: Inserimento con cicli di lettura/scrittura

3.1.1 Obiettivo

Questo test verifica il comportamento quando due transazioni leggono contemporaneamente un valore (il massimo CFU presente nella tabella) e lo utilizzano per un nuovo inserimento. Lo scopo è evidenziare anomalie come il *write skew*, che si verificano quando due transazioni basano le loro decisioni di scrittura su letture sovrapposte.

3.1.2 Procedura

1. Due client avviano transazioni con lo stesso livello di isolamento
2. Entrambi leggono il valore massimo di CFU presente nella tabella
3. Il primo client inserisce un nuovo studente con CFU pari al massimo letto + 10
4. Il secondo client inserisce un nuovo studente con CFU pari al massimo letto + 5
5. Entrambi i client tentano di committare le transazioni

3.1.3 Risultati attesi

3.1.4 Risultati attesi

- Con `READ COMMITTED` e `REPEATABLE READ`, entrambe le transazioni dovrebbero completarsi con successo. Sebbene `REPEATABLE READ` offra maggiori garanzie rispetto al suo predecessore, non è progettato per prevenire tutte le anomalie di serializzazione come il *write skew*, che è ciò che questo test intende evidenziare.
- Con `SERIALIZABLE`, ci aspettiamo che una delle due transazioni fallisca con un errore di serializzazione, poiché il sistema rileva il potenziale conflitto.

3.2 Test 2: Modifica concorrentiale dello stesso campo

3.2.1 Obiettivo

Questo test verifica il comportamento quando due transazioni tentano di modificare contemporaneamente lo stesso campo dello stesso record. Lo scopo è evidenziare come PostgreSQL gestisce i conflitti diretti di update.

3.2.2 Procedura

1. Due client avviano transazioni con lo stesso livello di isolamento
2. Entrambi leggono il valore di CFU di uno studente specifico
3. Il primo client aggiorna il CFU incrementandolo di 10
4. Il secondo client tenta di aggiornare lo stesso CFU incrementandolo di 5
5. Entrambi i client tentano di committare le transazioni

3.2.3 Risultati attesi

- Con tutti i livelli di isolamento, ci aspettiamo che il secondo client rimanga in attesa che il primo rilasci il lock sulla riga
- Se il primo client committa, il secondo dovrebbe poi essere in grado di applicare la sua modifica
- Se impostato un timeout, il secondo client potrebbe andare in timeout se il primo non committa in tempo

3.3 Test 3: Inserimento di un campo chiave primaria già esistente

3.3.1 Obiettivo

Questo test verifica il comportamento quando due transazioni tentano di inserire contemporaneamente un record con la stessa chiave primaria. Lo scopo è evidenziare come PostgreSQL gestisce i conflitti di vincoli unici.

3.3.2 Procedura

1. Due client avviano transazioni con lo stesso livello di isolamento
2. Entrambi verificano che non esista uno studente con una specifica matricola
3. Il primo client inserisce uno studente con quella matricola
4. Il secondo client tenta di inserire un altro studente con la stessa matricola
5. Entrambi i client tentano di committare le transazioni

3.3.3 Risultati attesi

- Con tutti i livelli di isolamento, ci aspettiamo che il secondo client rimanga in attesa che il primo rilasci il lock sull'indice della chiave primaria
- Se il primo client committa, il secondo dovrebbe fallire con un errore di violazione del vincolo di unicità
- Se impostato un timeout, il secondo client potrebbe andare in timeout se il primo non committa in tempo

4 Implementazione

4.1 Connessione al database

Nota sulla metodologia: Per semplicità didattica, l'esecuzione concorrente è stata simulata eseguendo le istruzioni dei due client in modo alternato all'interno di un unico script Python. Questo approccio è sufficiente per dimostrare i fenomeni di locking e serializzazione, anche se in un'applicazione reale i client opererebbero in thread o processi indipendenti. Per prima cosa sono stati stabiliti due client distinti connessi allo stesso database:

```
1 # Connessione del primo client al database
2 conn1 = psycopg2.connect(
3     dbname="postgres",
4     user="postgres",
5     password="postgres",
6     host="localhost",
7     port="5432"
8 )
9
10 # Connessione del secondo client al database
11 conn2 = psycopg2.connect(
12     dbname="postgres",
13     user="postgres",
14     password="postgres",
15     host="localhost",
16     port="5432"
17 )
18
19 # Cursore per interrogazioni per il primo client
20 cursor1 = conn1.cursor()
21
22 # Cursore per interrogazioni per il secondo client
23 cursor2 = conn2.cursor()
```

4.2 Implementazione del Test 1

Il codice per il test di inserimento con cicli di lettura/scrittura è il seguente:

```
1 def test_read_write_cycle(isolation_level):
2     print(f"\n=== Test con livello di isolamento {isolation_level} ===")
3
4     # Impostazione del livello di isolamento e avvio transazione per
4     client 1
5     cursor1.execute(f"START TRANSACTION ISOLATION LEVEL {isolation_level}");
6
7     # Client 1 legge il massimo CFU esistente
8     cursor1.execute("SELECT MAX(cfu) FROM Studente;")
```

```

9     max_cfu1 = cursor1.fetchone()[0]
10    print(f"Client 1: Massimo CFU rilevato = {max_cfu1}")
11
12    # Impostazione del livello di isolamento e avvio transazione per
client 2
13    cursor2.execute(f"START TRANSACTION ISOLATION LEVEL {isolation_level
};")
14
15    # Client 2 legge il massimo CFU esistente
16    cursor2.execute("SELECT MAX(cfu) FROM Studente;")
17    max_cfu2 = cursor2.fetchone()[0]
18    print(f"Client 2: Massimo CFU rilevato = {max_cfu2}")
19
20    try:
21        # Client 1 inserisce un nuovo studente con CFU pari al massimo +
10
22        new_cfu1 = max_cfu1 + 10
23        cursor1.execute(f"""
24            INSERT INTO Studente (matricola, nome, cognome, cfu)
25            VALUES ('99001', 'Mario', 'Rossi', {new_cfu1});
26        """)
27        print(f"Client 1: Inserito studente con {new_cfu1} CFU")
28
29        # Client 2 inserisce un nuovo studente con CFU pari al massimo +
5
30        new_cfu2 = max_cfu2 + 5
31        cursor2.execute(f"""
32            INSERT INTO Studente (matricola, nome, cognome, cfu)
33            VALUES ('99002', 'Laura', 'Bianchi', {new_cfu2});
34        """)
35        print(f"Client 2: Inserito studente con {new_cfu2} CFU")
36
37        # Commit delle transazioni
38        conn1.commit()
39        print("Client 1: Commit eseguito con successo")
40
41        conn2.commit()
42        print("Client 2: Commit eseguito con successo")
43
44    except Exception as e:
45        print(f"Errore: {e}")
46        conn1.rollback()
47        conn2.rollback()
48
49    # Verifica finale
50    cursor1.execute("SELECT matricola, cfu FROM Studente WHERE matricola
IN ('99001', '99002') ORDER BY cfu DESC;")
51    result = cursor1.fetchall()
52    print("\nRisultato finale:")
53    for row in result:
54        print(f"Matricola: {row[0]}, CFU: {row[1]}")

```

4.3 Implementazione del Test 2

Il codice per il test di modifica concorrentiale dello stesso campo è il seguente:

```

1 def test_concurrent_update(isolation_level):
2     print(f"\n=== Test con livello di isolamento {isolation_level} ===")
3
4     # Preparo il record di test
5     cursor1.execute("DELETE FROM Studente WHERE matricola = '10001';")
6     cursor1.execute("INSERT INTO Studente (matricola, nome, cognome, cfu
7 ) VALUES ('10001', 'Test', 'User', 60);")
8     conn1.commit()
9
10    # Impostazione del livello di isolamento e avvio transazione per
    client 1
11    cursor1.execute(f"START TRANSACTION ISOLATION LEVEL {isolation_level
12 };")
13
14    # Imposto un timeout di 3 secondi per evitare blocchi infiniti
15    cursor2.execute("SET statement_timeout = 3000;") # 3000 ms = 3
    secondi
16
17    # Impostazione del livello di isolamento e avvio transazione per
    client 2
18    cursor2.execute(f"START TRANSACTION ISOLATION LEVEL {isolation_level
19 };")
20
21    try:
22        # Client 1 legge il record
23        cursor1.execute("SELECT cfu FROM Studente WHERE matricola =
24 '10001';")
25        cfu1 = cursor1.fetchone()[0]
26        print(f"Client 1: Letto CFU = {cfu1}")
27
28        # Client 2 legge il record
29        cursor2.execute("SELECT cfu FROM Studente WHERE matricola =
30 '10001';")
31        cfu2 = cursor2.fetchone()[0]
32        print(f"Client 2: Letto CFU = {cfu2}")
33
34        # Client 1 aggiorna il record
35        cursor1.execute("UPDATE Studente SET cfu = cfu + 10 WHERE
36 matricola = '10001';")
37        print("Client 1: Aggiornati CFU = cfu + 10")
38
39        # Client 2 aggiorna il record - Questo potrebbe bloccarsi
40        try:
41            cursor2.execute("UPDATE Studente SET cfu = cfu + 5 WHERE
42 matricola = '10001';")
43            print("Client 2: Aggiornati CFU = cfu + 5")
44        except psycopg2.errors.QueryCanceled as e:
45            print("Client 2: Timeout raggiunto durante l'attesa del lock
46 ")
47
48            print("Questo      un comportamento normale in PostgreSQL
49 quando pi  client cercano di modificare lo stesso record")
50            raise e
51
52        # Commit delle transazioni
53        conn1.commit()
54        print("Client 1: Commit eseguito con successo")
55
56        conn2.commit()

```

```

47         print("Client 2: Commit eseguito con successo")
48
49     except Exception as e:
50         print(f"Errore: {e}")
51         # Assicuriamoci di fare rollback di entrambe le connessioni
52         try:
53             conn1.rollback()
54             print("Client 1: Rollback eseguito")
55         except:
56             pass
57         try:
58             conn2.rollback()
59             print("Client 2: Rollback eseguito")
60         except:
61             pass
62     finally:
63         # Reset del timeout alla fine del test
64         cursor2.execute("SET statement_timeout = 0;") # Torniamo al
        valore predefinito
65
66     # Verifica finale
67     cursor1.execute("SELECT matricola, cfu FROM Studente WHERE matricola
        = '10001';")
68     result = cursor1.fetchone()
69     if result:
70         print(f"\nRisultato finale: Matricola {result[0]}, CFU: {result
        [1]}")
71     else:
72         print("\nRisultato finale: Record non trovato")

```

4.4 Implementazione del Test 3

Il codice per il test di inserimento di un campo chiave primaria già esistente è il seguente:

```

1 def test_primary_key_conflict(isolation_level):
2     print(f"\n=== Test con livello di isolamento {isolation_level} ===")
3
4     # Rimuovo eventuali record di test precedenti
5     cursor1.execute("DELETE FROM Studente WHERE matricola = '20001';")
6     conn1.commit()
7
8     # Impostazione del livello di isolamento e avvio transazione per
    client 1
9     cursor1.execute(f"START TRANSACTION ISOLATION LEVEL {isolation_level
    };")
10
11     # Imposto un timeout di 3 secondi per evitare blocchi infiniti
12     cursor2.execute("SET statement_timeout = 3000;") # 3000 ms = 3
    secondi
13
14     # Impostazione del livello di isolamento e avvio transazione per
    client 2
15     cursor2.execute(f"START TRANSACTION ISOLATION LEVEL {isolation_level
    };")
16
17     try:

```

```

18     # Client 1 verifica se la matricola esiste
19     cursor1.execute("SELECT COUNT(*) FROM Studente WHERE matricola =
20     '20001';")
21     count1 = cursor1.fetchone()[0]
22     print(f"Client 1: La matricola 20001 {'esiste gi ' if count1 >
0 else 'non esiste'}")
23
24     # Client 2 verifica se la matricola esiste
25     cursor2.execute("SELECT COUNT(*) FROM Studente WHERE matricola =
26     '20001';")
27     count2 = cursor2.fetchone()[0]
28     print(f"Client 2: La matricola 20001 {'esiste gi ' if count2 >
0 else 'non esiste'}")
29
30     # Client 1 inserisce un nuovo studente con la matricola 20001
31     cursor1.execute("""
32     INSERT INTO Studente (matricola, nome, cognome, cfu)
33     VALUES ('20001', 'Franco', 'Neri', 100);
34     """)
35     print("Client 1: Inserito studente con matricola 20001")
36
37     # Client 2 inserisce un nuovo studente con la stessa matricola
38     20001
39     try:
40         cursor2.execute("""
41         INSERT INTO Studente (matricola, nome, cognome, cfu)
42         VALUES ('20001', 'Anna', 'Verdi', 120);
43         """)
44         print("Client 2: Inserito studente con matricola 20001")
45     except psycopg2.errors.QueryCanceled as e:
46         print("Client 2: Timeout raggiunto durante l'attesa del lock
47 ")
48         print("Questo un comportamento normale quando pi client
49 cercano di inserire lo stesso valore chiave")
50         raise e
51     except psycopg2.errors.UniqueViolation as e:
52         print("Client 2: Violazione del vincolo di chiave primaria")
53         print("Questo errore si verifica quando il primo client ha
54 gi committato l'inserimento")
55         raise e
56
57     # Commit delle transazioni
58     conn1.commit()
59     print("Client 1: Commit eseguito con successo")
60
61     conn2.commit()
62     print("Client 2: Commit eseguito con successo")
63
64 except Exception as e:
65     print(f"Errore: {e}")
66     # Assicuriamoci di fare rollback di entrambe le connessioni
67     try:
68         conn1.rollback()
69         print("Client 1: Rollback eseguito")
70     except:
71         pass
72     try:
73         conn2.rollback()

```

```
68         print("Client 2: Rollback eseguito")
69     except:
70         pass
71     finally:
72         # Reset del timeout alla fine del test
73         cursor2.execute("SET statement_timeout = 0;") # Torniamo al
valore predefinito
74
75     # Verifica finale
76     cursor1.execute("SELECT matricola, nome, cognome, cfu FROM Studente
WHERE matricola = '20001';")
77     result = cursor1.fetchall()
78     print("\nRisultato finale:")
79     if result:
80         for row in result:
81             print(f"Matricola: {row[0]}, Nome: {row[1]}, Cognome: {row
[2]}, CFU: {row[3]}")
82     else:
83         print("Nessun record trovato con matricola 20001")
```

5 Risultati ottenuti

5.1 Risultati del Test 1

5.1.1 READ COMMITTED

```
=== Test con livello di isolamento READ COMMITTED ===
Client 1: Massimo CFU rilevato = 168
Client 2: Massimo CFU rilevato = 168
Client 1: Inserito studente con 178 CFU
Client 2: Inserito studente con 173 CFU
Client 1: Commit eseguito con successo
Client 2: Commit eseguito con successo
```

Risultato finale:
Matricola: 99001, CFU: 178
Matricola: 99002, CFU: 173

5.1.2 REPEATABLE READ

```
=== Test con livello di isolamento REPEATABLE READ ===
Client 1: Massimo CFU rilevato = 168
Client 2: Massimo CFU rilevato = 168
Client 1: Inserito studente con 178 CFU
Client 2: Inserito studente con 173 CFU
Client 1: Commit eseguito con successo
Client 2: Commit eseguito con successo
```

Risultato finale:
Matricola: 99001, CFU: 178
Matricola: 99002, CFU: 173

5.1.3 SERIALIZABLE

```
=== Test con livello di isolamento SERIALIZABLE ===
Client 1: Massimo CFU rilevato = 168
Client 2: Massimo CFU rilevato = 168
Client 1: Inserito studente con 178 CFU
Client 2: Inserito studente con 173 CFU
Client 1: Commit eseguito con successo
Errore: ERRORE: serializzazione dell'accesso fallita a causa di dipendenze di lettura/scrittura
DETAIL: Reason code: Canceled on identification as a pivot, during commit attempt.
HINT: La transazione potrebbe riuscire se ritentata.
```

Risultato finale:

Matricola: 99001, CFU: 178

5.2 Risultati del Test 2

5.2.1 READ COMMITTED

=== Test con livello di isolamento READ COMMITTED ===

Client 1: Letto CFU = 60

Client 2: Letto CFU = 60

Client 1: Aggiornati CFU = cfu + 10

Client 2: Timeout raggiunto durante l'attesa del lock

Questo è un comportamento normale in PostgreSQL quando più client cercano di modificare lo st

Errore: ERRORE: annullamento dell'istruzione a causa di timeout

CONTEXT: durante la modifica della tupla (0,57) nella relazione "studente"

Client 1: Rollback eseguito

Client 2: Rollback eseguito

Risultato finale: Matricola 10001, CFU: 60

5.2.2 REPEATABLE READ

=== Test con livello di isolamento REPEATABLE READ ===

Client 1: Letto CFU = 60

Client 2: Letto CFU = 60

Client 1: Aggiornati CFU = cfu + 10

Client 2: Timeout raggiunto durante l'attesa del lock

Questo è un comportamento normale in PostgreSQL quando più client cercano di modificare lo st

Errore: ERRORE: annullamento dell'istruzione a causa di timeout

CONTEXT: durante la modifica della tupla (0,59) nella relazione "studente"

Client 1: Rollback eseguito

Client 2: Rollback eseguito

Risultato finale: Matricola 10001, CFU: 60

5.2.3 SERIALIZABLE

=== Test con livello di isolamento SERIALIZABLE ===

Client 1: Letto CFU = 60

Client 2: Letto CFU = 60

Client 1: Aggiornati CFU = cfu + 10

Client 2: Timeout raggiunto durante l'attesa del lock

Questo è un comportamento normale in PostgreSQL quando più client cercano di modificare lo st

Errore: ERRORE: annullamento dell'istruzione a causa di timeout

CONTEXT: durante la modifica della tupla (0,61) nella relazione "studente"

Client 1: Rollback eseguito

Client 2: Rollback eseguito

Risultato finale: Matricola 10001, CFU: 60

5.3 Risultati del Test 3

5.3.1 READ COMMITTED

=== Test con livello di isolamento READ COMMITTED ===

Client 1: La matricola 20001 non esiste

Client 2: La matricola 20001 non esiste

Client 1: Inserito studente con matricola 20001

Client 2: Timeout raggiunto durante l'attesa del lock

Questo è un comportamento normale quando più client cercano di inserire lo stesso valore chiave

Errore: ERRORE: annullamento dell'istruzione a causa di timeout

CONTEXT: durante l'inserimento della tupla di indice (0,64) nella relazione "studente_pkey"

Client 1: Rollback eseguito

Client 2: Rollback eseguito

Risultato finale:

Nessun record trovato con matricola 20001

5.3.2 REPEATABLE READ

=== Test con livello di isolamento REPEATABLE READ ===

Client 1: La matricola 20001 non esiste

Client 2: La matricola 20001 non esiste

Client 1: Inserito studente con matricola 20001

Client 2: Timeout raggiunto durante l'attesa del lock

Questo è un comportamento normale quando più client cercano di inserire lo stesso valore chiave

Errore: ERRORE: annullamento dell'istruzione a causa di timeout

CONTEXT: durante l'inserimento della tupla di indice (0,66) nella relazione "studente_pkey"

Client 1: Rollback eseguito

Client 2: Rollback eseguito

Risultato finale:

Nessun record trovato con matricola 20001

5.3.3 SERIALIZABLE

=== Test con livello di isolamento SERIALIZABLE ===

Client 1: La matricola 20001 non esiste

Client 2: La matricola 20001 non esiste

Client 1: Inserito studente con matricola 20001

Client 2: Timeout raggiunto durante l'attesa del lock

Questo è un comportamento normale quando più client cercano di inserire lo stesso valore chiave

Errore: ERRORE: annullamento dell'istruzione a causa di timeout

5 Risultati ottenuti

CONTEXT: durante l'inserimento della tupla di indice (0,68) nella relazione "studente_pkey"

Client 1: Rollback eseguito

Client 2: Rollback eseguito

Risultato finale:

Nessun record trovato con matricola 20001

6 Analisi dei risultati

6.1 Test 1: Inserimento con cicli di lettura/scrittura

- **READ COMMITTED e REPEATABLE READ:** Come previsto, entrambi i client hanno letto lo stesso valore massimo di CFU (168) e sono riusciti a inserire i loro record con CFU incrementati (178 e 173). Entrambe le transazioni sono state committate con successo. Questo conferma che questi livelli di isolamento non rilevano anomalie di tipo write skew, dove due transazioni basano le loro decisioni di scrittura su letture sovrapposte.
- **SERIALIZABLE:** In linea con le aspettative, mentre il primo client ha inserito e committato con successo il suo record, il secondo client ha ricevuto un errore di serializzazione durante il tentativo di commit. L'errore specifica chiaramente il motivo: "serializzazione dell'accesso fallita a causa di dipendenze di lettura/scrittura tra le transazioni". Questo dimostra che il livello SERIALIZABLE rileva correttamente il potenziale conflitto di serializzazione e interrompe una delle transazioni per garantire la consistenza.

6.2 Test 2: Modifica concorrente dello stesso campo

Per tutti i livelli di isolamento, le esecuzioni iniziali del test senza un meccanismo di timeout hanno portato a un blocco indefinito dello script. Questo fenomeno si è verificato perché, dopo che il primo client ha ottenuto un lock esclusivo sulla riga tramite l'operazione di `UPDATE`, il secondo client è stato messo correttamente in attesa dal database. A sua volta, lo script di test, essendo bloccato sull'esecuzione del secondo client, non poteva procedere al commit del primo, creando un *deadlock a livello applicativo*.

L'introduzione di un `statement_timeout` di 3 secondi si è rivelata una soluzione strategica. Come mostrato nei risultati, questo ha trasformato il blocco indefinito in un errore `QueryCanceled` controllato. Questo risultato dimostra in modo inequivocabile il meccanismo di lock a livello di riga di PostgreSQL: qualsiasi transazione che tenta di modificare una riga già bloccata deve attendere. L'esito uniforme tra i vari livelli di isolamento evidenzia che, in scenari di conflitto diretto su una risorsa (*write-write conflict*), il meccanismo di locking prevale sulle regole di visibilità dell'MVCC.

6.3 Test 3: Inserimento di un campo chiave primaria già esistente

Analogamente al test precedente, anche in questo caso il risultato è stato un timeout per il secondo client, consistente per tutti i livelli di isolamento.

- Entrambi i client hanno verificato che la matricola 20001 non esistesse nel loro rispettivo snapshot.
- Il primo client ha proceduto con l'inserimento, ottenendo un lock esclusivo sulla nuova chiave nell'indice primario (`studente_pkey`).

- Il secondo client, tentando di inserire la stessa chiave, è stato messo in coda, in attesa del rilascio del lock.
- L'impostazione del timeout ha interrotto l'attesa, causando un fallimento controllato della seconda transazione.

Questo test conferma che PostgreSQL utilizza un meccanismo di locking robusto anche a livello di indice per garantire l'unicità delle chiavi. Se il primo client avesse committato, il secondo avrebbe ricevuto immediatamente un errore `UniqueViolation`. Il timeout ha quindi agito come meccanismo di sicurezza contro un'attesa potenzialmente infinita, un aspetto fondamentale nella progettazione di applicazioni concorrenti robuste.

7 Conclusioni

I test effettuati hanno dimostrato chiaramente il diverso comportamento di PostgreSQL nei vari livelli di isolamento. Di seguito sono riportate le principali conclusioni:

7.1 Differenze tra i livelli di isolamento

- **READ COMMITTED**: Fornisce il livello minimo di isolamento ma con la massima concorrenza. Non rileva anomalie complesse come write skew, ma è sufficiente per molti casi d'uso. È il livello predefinito in PostgreSQL proprio perché offre un buon equilibrio tra consistenza e performance.
- **REPEATABLE READ**: Offre maggiori garanzie rispetto al READ COMMITTED, assicurando che le letture ripetute restituiscano sempre gli stessi valori all'interno di una transazione. Tuttavia, come abbiamo visto nel Test 1, non rileva anomalie di tipo write skew.
- **SERIALIZABLE** dovrebbe essere usato quando è essenziale prevenire tutte le anomalie di concorrenza, garantendo che il risultato dell'interazione tra transazioni sia logicamente equivalente a un'esecuzione seriale.

7.2 Comportamento di lock in PostgreSQL

I Test 2 e 3 hanno evidenziato il meccanismo di lock utilizzato da PostgreSQL:

- I lock vengono acquisiti sia sulle righe (per le operazioni di UPDATE) che sugli indici (per le operazioni che coinvolgono chiavi primarie e indici unici)
- Quando una transazione detiene un lock su una riga o un indice, altre transazioni che tentano di accedere alla stessa risorsa devono attendere il rilascio del lock
- Implementare timeout appropriati è essenziale per evitare blocchi indefiniti

7.3 Considerazioni per le applicazioni reali

Sulla base dei risultati ottenuti, possiamo formulare alcune raccomandazioni per le applicazioni reali:

- La scelta del livello di isolamento dovrebbe essere basata sulle specifiche esigenze dell'applicazione, bilanciando consistenza e performance
- READ COMMITTED è spesso sufficiente per applicazioni che non richiedono garanzie di serializzazione stringenti

7 Conclusioni

- `SERIALIZABLE` dovrebbe essere usato quando è essenziale garantire che tutte le transazioni operino su uno snapshot consistente del database
- È importante implementare una logica di retry per gestire gli errori di serializzazione che possono verificarsi nel livello `SERIALIZABLE`
- Per evitare deadlock o attese indefinite, è consigliabile impostare timeout appropriati sulle transazioni

Il sistema MVCC di PostgreSQL si dimostra estremamente efficace nel massimizzare la concorrenza, specialmente per carichi di lavoro a prevalenza di letture. La combinazione di snapshot non bloccanti e un meccanismo di locking granulare per le scritture, unito alla flessibilità offerta dai diversi livelli di isolamento, rende PostgreSQL una scelta eccellente per applicazioni transazionali complesse e ad alte prestazioni.